TOSS BANK

Server Developer Intern Portfolio

문제를 기능 목록으로만 보지 않고,
사용자·데이터·성능·운영의 흐름으로 풀었습니다.

STUDY TRACKER API

Java · Spring Boot · JPA · PostgreSQL · Redis
Security · Test · Docker · GitHub Actions · Render

[이름 입력]
Server Developer Candidate

2026.08
GitHub · Live API

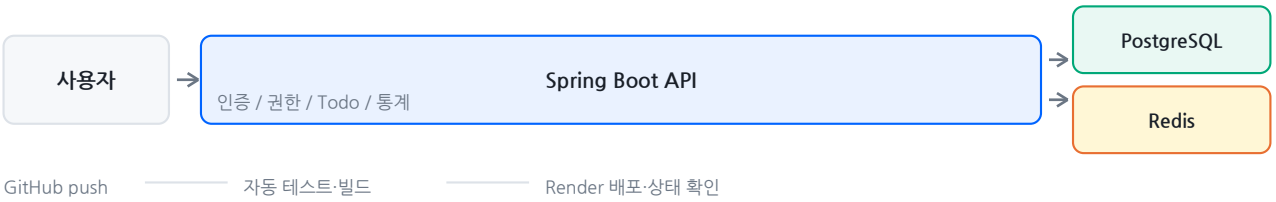
01 프로젝트 한눈에

요구사항부터 배포 후 확인까지, 하나의 서비스 흐름으로 완성했습니다.

Study Tracker API는 사용자가 학습 Todo를 안전하게 관리하는 서버입니다. CRUD를 만드는 데서 멈추지 않고, 서비스가 실제로 운영될 때 필요한 로그인, 사용자별 데이터 보호, 데이터 저장, 반복 조회 개선, 중복 요청 방지, 자동 테스트와 배포, 장애 확인 절차까지 범위를 넓혔습니다.

49	14개	2곳	운영
자동 테스트	주요 API	PostgreSQL + Redis	상태 확인 주소

전체 구조



요구사항을 이렇게 나눴습니다

사용자

회원가입·로그인
토큰 재발급·로그아웃
비밀번호 암호화

핵심 기능

Todo 생성·조회·수정·완료·삭제
검색·완료 여부 필터
통계 조회

신뢰성

다른 사용자 데이터 접근 차단
같은 생성 요청 중복 처리 방지
일관된 에러 응답

운영

자동 테스트·빌드
Docker·Render 배포
상태 확인·장애 점검 문서

02 다른 사용자의 Todo를 어떻게 지킬까

기능을 추가하기 전에, 데이터 소유권을 모든 요청의 기본 조건으로 만들었습니다.

문제 정의

Todo 번호만 맞으면 조회·수정·삭제가 되는 구조에서는, 번호를 추측하거나 우연히 알게 된 사용자가 다른 사람의 데이터를 건드릴 수 있습니다. '로그인했는가'와 '그 데이터의 주인인가'는 별개의 문제라고 정의했습니다.

01 요구사항

모든 Todo 작업은 현재 로그인한 사용자의 데이터만 다룬다.

→

02 설계

Todo 번호와 사용자 번호를 함께 검색 조건에 넣는다.

→

03 응답

타인의 Todo는 존재 여부도 드러내지 않고 찾을 수 없음으로 처리한다.

왜 이 방식을 선택했나

- 화면이나 주소 단계에서 막는 것만으로는 API 직접 호출을 막을 수 없으므로 서버의 데이터 조회 조건에 소유권을 포함했습니다.
- 조회 뒤 사용자 정보를 비교하는 방식보다, 처음부터 'Todo 번호 + 사용자 번호'로 조회하면 실수로 검사를 빼먹을 가능성을 줄일 수 있습니다.
- 타인의 데이터에 403 대신 404를 반환해 해당 번호의 Todo가 실제로 존재하는지 노출하지 않았습니다.

어떻게 검증했나

검증 상황	기대 결과
다른 사용자가 Todo 목록 조회	소유자의 Todo가 보이지 않음
다른 사용자가 단건 조회	404 응답
다른 사용자가 제목 수정	404 응답, 원본 유지
다른 사용자가 삭제	404 응답, 소유자는 계속 조회 가능

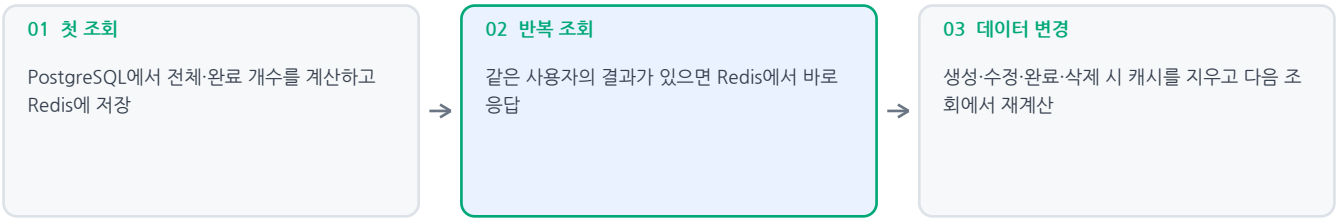
배운 점

권한은 로그인 여부를 확인하는 한 번의 기능이 아니라, 데이터가 들어오고 나가는 모든 경로에 반복해서 적용되는 설계 조건입니다. 테스트도 '내 요청이 성공하는가'뿐 아니라 '남의 요청이 실패하고 원본이 보존되는가'까지 작성해야 합니다.

03 반복 통계 조회를 어떻게 줄일까

같은 계산의 반복을 Redis로 줄이되, 오래된 값이 남지 않도록 변경 흐름까지 설계했습니다.

Todo 통계는 전체 개수와 완료 개수를 보여줍니다. 값이 바뀌지 않았는데도 화면을 열 때마다 데이터베이스에서 두 번 집계하는 것은 반복 비용입니다. 특히 사용자가 늘면 같은 형태의 집계 요청도 함께 늘어납니다.



캐시 유지 시간: 60초

예상치 못한 무효화 누락이 있어도 오래된 값이 무기한 남지 않도록 제한했습니다.

캐시 단위: 사용자

사용자마다 통계가 다르므로 사용자 번호를 기준으로 결과를 분리했습니다.

데이터베이스 조회 조건도 함께 정리

Todo 목록과 통계는 항상 사용자 기준으로 조회하고, 완료 여부를 자주 조건으로 사용합니다. 그래서 사용자 번호를 첫 기준으로 두고 Todo 번호 또는 완료 여부를 이어 붙인 인덱스를 설계했습니다.

조회 목적	인덱스 구성
사용자의 Todo 목록	사용자 번호 + Todo 번호
사용자의 완료 여부 필터	사용자 번호 + 완료 여부 + Todo 번호

검증 결과

첫 조회 뒤 데이터베이스에 Todo를 직접 추가해도 두 번째 조회가 저장된 통계를 반환하는지 확인했습니다. 이어 Todo 생성 기능을 통해 데이터를 바꾼 뒤에는 캐시가 삭제되어 새 통계가 계산되는지 자동 테스트로 검증했습니다. 아직 운영 환경의 응답 시간을 수치로 측정하지 않았으므로, 확인하지 않은 성능 배수는 주장하지 않습니다.

04 같은 생성 요청을 어떻게 한 번만 처리할까

사용자 실수와 네트워크 재시도를 서버가 견딜 수 있도록 요청 자체에 식별값을 붙였습니다.

발생 가능한 상황

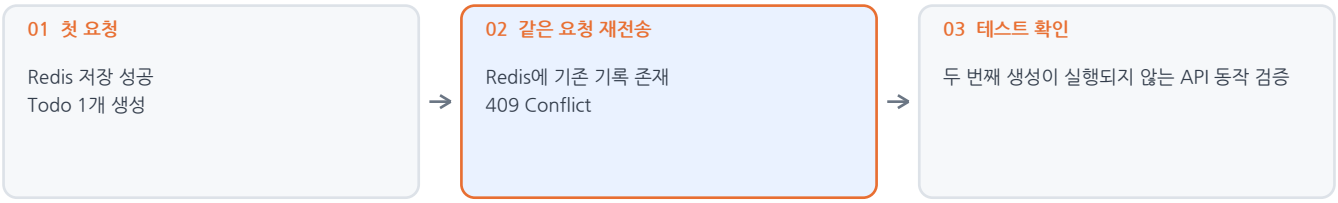
사용자가 생성 버튼을 두 번 누르거나, 응답을 받지 못한 클라이언트가 같은 요청을 다시 보낼 수 있습니다. 서버가 모든 요청을 새 요청으로 받아들이면 같은 Todo가 두 개 생깁니다.

처리 흐름

1	클라이언트가 생성 요청마다 고유한 요청 식별값을 함께 보냅니다.
2	서버는 사용자 번호·작업 이름·요청 식별값을 묶어 Redis에 저장을 시도합니다.
3	처음 온 요청만 저장에 성공하고 Todo를 생성합니다. 같은 요청은 409 응답으로 거절합니다.
4	처리 도중 실패하면 식별값을 지워 정상적인 재시도가 가능하게 하고, 성공 기록은 5분 후 자동 삭제합니다.

설계에서 고민한 경계

- 사용자가 다르면 같은 요청 식별값을 사용해도 서로 영향을 주지 않도록 사용자 번호를 저장 키에 포함했습니다.
- 저장에 성공한 뒤 실제 처리에서 오류가 나면 기록을 삭제해, 실패 요청까지 완료된 것으로 막지 않게 했습니다.
- Redis의 '값이 없을 때만 저장' 기능을 사용해, 동시에 같은 요청이 와도 하나만 시작하도록 했습니다.



한계와 다음 개선

현재는 중복 요청에 이전 성공 응답을 다시 돌려주지 않고 409로 거절합니다. 결제처럼 재시도 결과까지 동일해야 하는 업무라면 최초 응답을 함께 저장하고, 요청 내용이 달라진 경우를 구분하는 설계가 더 필요합니다.

05 관계 조회 문제를 어떻게 확인하고 줄였나

지연 조회의 장점은 유지하고, 여러 건을 함께 읽는 화면에서만 필요한 데이터를 한 번에 가져왔습니다.

Todo는 사용자를 참조하고, 사용자는 여러 Todo를 가집니다. 평소에는 연관 데이터를 바로 읽지 않아 불필요한 조회를 피할 수 있지만, 모든 사용자와 각 사용자의 Todo를 함께 보여주는 상황에서는 사용자마다 추가 쿼리가 발생할 수 있습니다.

일반 관계 조회	필요한 관계를 함께 조회
사용자 목록 1회 조회 뒤 각 사용자의 Todo를 읽을 때 추가 조회	사용자와 Todo를 첫 조회에서 함께 가져와 뒤따르는 추가 조회를 없앰
2명 테스트: 1회보다 많은 쿼리	2명 테스트: 1회

검증 방법

01 **테스트 데이터 준비**
사용자 2명과 각 사용자의 Todo 1개를 저장하고, 이미 읽힌 데이터의 영향을 없애기 위해 조회 상태를 초기화했습니다.

02 **실제 쿼리 수 기록**
데이터베이스 조회 통계를 켜고 일반 관계 조회와 개선 조회가 실행한 쿼리 수를 각각 기록했습니다.

03 **결과를 조건으로 고정**
일반 조회는 1회보다 많고, 관계를 함께 가져오는 조회는 정확히 1회인지 자동 테스트의 성공 조건으로 만들었습니다.

왜 모든 관계를 항상 한 번에 읽지 않았나

모든 API가 사용자와 Todo를 동시에 필요로 하는 것은 아닙니다. 항상 함께 읽으면 필요하지 않은 데이터까지 가져올 수 있습니다. 기본 관계는 필요할 때 읽도록 두고, 목록처럼 함께 필요한 사용 사례에만 전용 조회를 적용했습니다.

테스트 전략

기능 단위	API 통합	데이터 저장
서비스 규칙과 예외를 빠르게 확인	로그인부터 요청·응답·권한까지 확인	사용자별 조회와 관계 쿼리 확인

06 배포 뒤에도 설명할 수 있는 서비스

코드가 실행되는 순간이 끝이 아니라, 상태를 확인하고 문제를 좁히는 과정까지 준비했습니다.

자동 테스트·빌드·배포 흐름

01 GitHub push

main 브랜치에 변경 사항을 올립니다.

→

02 자동 검증

Java 21 환경에서 49개 테스트와 실행 파일 빌드를 수행합니다.

→

03 Render 배포

Docker 이미지로 서버를 실행하고 PostgreSQL·Redis에 연결합니다.

배포 이후 확인 항목

확인 대상	확인 방법	판단
서버	상태 확인 API	정상 실행 여부
데이터베이스	상태 응답·Render 로그	연결 실패 여부
Redis	상태 응답·Render 로그	캐시 연결 여부
배포	이벤트·최근 커밋	어떤 변경이 반영됐는지
설정	환경변수 목록	누락·오타 여부

장애를 만났을 때의 순서

Render 로그 확인 -> 최근 배포 커밋 확인 -> 서버 상태 응답 확인 -> PostgreSQL 연결 확인 -> Redis 연결 확인 -> 환경변수 누락 확인

운영 경험을 문서로 남긴 이유

장애가 발생한 순간에는 기억보다 순서가 중요합니다. 상태 확인 주소, 자주 발생하는 증상과 원인, 배포 뒤 확인 명령을 운영 문서로 정리해 같은 문제를 다시 만났을 때 빠르게 범위를 좁힐 수 있도록 했습니다.

현재 확인 가능한 주소

API 서버 study-tracker-api-7stm.onrender.com
API 문서 Swagger UI
상태 확인 /actuator/health

07 토스뱅크에서 이어갈 성장

지금까지 만든 기반을 더 높은 확장성·가용성·관측 가능성의 문제로 넓히고 싶습니다.

이 프로젝트에서 가장 중요하게 배운 것은 기술 이름보다 문제를 구조로 바꾸는 과정입니다. '사용자는 자기 데이터만 본다'를 데이터 조회 조건과 테스트로, '반복 계산을 줄인다'를 캐시 저장·삭제 흐름으로, '배포 뒤 상태를 안다'를 자동 검증과 운영 문서로 구체화했습니다.

토스뱅크가 찾는 경험	프로젝트에서 보여준 근거
요건을 데이터 모델과 API로 풀어본 경험	사용자·Todo·로그인 갱신 정보를 관계로 설계하고 14개 주요 API로 구현
성능 최적화에 대한 고민	통계 캐시, 변경 시 캐시 삭제, 조회 조건 기반 인덱스, 관계 조회 쿼리 개선
자동화와 운영에 대한 고민	49개 테스트, GitHub Actions, Docker·Render 배포, 상태 확인과 장애 점검 문서
확장 가능한 구조에 대한 관심	서버 내부 상태를 줄인 토큰 인증, PostgreSQL·Redis 분리, 계층별 역할 분리

특히 연결되는 조직

Product Foundation

내부 사용자가 반복해서 믿고 쓸 수 있는 API·플랫폼 품질에 관심이 있습니다.

Platform Engineering

장애를 확인하고 복구 시간을 줄이는 자동화·운영 구조를 더 깊게 배우고 싶습니다.

다음 성장 계획

01 언어

Java에서 익힌 구조를 Kotlin으로 옮기며 언어 차이를 설명

→

02 확장성

Kafka로 비동기 이벤트 흐름을 만들고 실패·재처리를 테스트

→

03 운영

부하 테스트와 Prometheus·Grafana 지표로 병목과 개선 결과를 수치화

[이름 입력]

justinnn980@gmail.com
GitHub · Project